

Organización de las Computadoras

2026

Temario

1- Modos de direccionamiento

2- Estructuras de Control

1- Modos de direccionamiento

Un modo de direccionamiento define cómo una instrucción específica sus operandos.

RISC-V utiliza cuatro modos principales:

- direccionamiento por registro,
- inmediato,
- base
- relativo al contador de programa (PC).

Los tres primeros modos definen los modos de lectura y escritura de operandos. El último modo define un modo de escritura del contador de programa (PC).

1.1- Modos de direccionamiento - por registro

El direccionamiento por registro, solo utiliza registros para todos los operandos de origen y destino.

Todas las instrucciones de tipo R utilizan direccionamiento por registro.

1.1- Modos de direccionamiento - por registro

add t2, t0, t1

1.2- Modos de direccionamiento - inmediato

El direccionamiento inmediato utiliza un valor inmediato, junto con registros, como operandos.

- Algunas instrucciones de tipo I, como “add immediate” (*addi*) utilizan direccionamiento inmediato con un valor inmediato con signo de 12 bits.
- También las instrucciones de desplazamiento con valor inmediato (*slli*, *srl* y *srai*) son instrucciones de tipo I que codifican el valor inmediato sin signo de 5 bits en imm4:0.
- Las instrucciones de carga (*lb*, *lh* y *lw*) utilizan el formato de instrucción de tipo I, pero emplean direccionamiento base, que se describe a continuación.

1.2- Modos de direccionamiento - inmediato

addi t2, t0, 5

1.3- Modos de direccionamiento - base (Indexado)

Las instrucciones de acceso a memoria, como cargar palabra (lw) y almacenar palabra (sw), utilizan direccionamiento base.

La dirección efectiva del operando de memoria se calcula sumando la dirección base del registro $rs1$ al desplazamiento de 12 bits con signo extendido que se encuentra en el campo inmediato.

Las cargas son instrucciones de tipo I y los almacenamientos son instrucciones de tipo S.

1.3- Modos de direccionamiento - base (Indexado)

$lw\ t2, \theta(t0)$

1.4- Modos de direccionamiento - relativo al PC

Las instrucciones de bifurcación, salto y enlace (*jal*) utilizan direccionamiento relativo al PC para especificar el nuevo valor del contador de programa (PC).

El desplazamiento con signo codificado en el campo inmediato se suma al PC para obtener la dirección de destino, el nuevo PC.

Los bits más significativos del desplazamiento se codifican en los campos inmediatos de 12 y 20 bits de las instrucciones de tipo B y J. El bit menos significativo del desplazamiento siempre es 0, por lo que no se codifica en la instrucción.

También la instrucción *auipc* (sumar valor inmediato superior al PC) utiliza direccionamiento relativo al PC.

1.4- Modos de direccionamiento - relativo al PC

```
beq t0, t1, etiqueta  
jal rd, etiqueta  
auipc s0, 0xABCD
```

1.5- Modos de direccionamiento - *Absoluto (directo)

Especifica una dirección de memoria.

No existe como instrucción en RISC-V, se usa una pseudo instrucción.

```
.data # segmento de datos
```

```
Label: 0x10348
```

```
.text # segmento de código
```

```
la t0, Label # auipc t0, 2 # segmento de datos 0x00002
```

```
# addi t0, t0, 0 # desplazamiento 0
```

```
lw s0, 0(t0) # lee el contenido de la dirección "absoluta"
```

2- Estructuras de Control

Las estructuras de control modifican el flujo del programa para que el procesador pueda obtener instrucciones que no están en orden secuencial en la memoria.

Por lo tanto modifican el PC para saltarse secciones de código o para repetir código anterior.

Existen dos tipos de estructuras de control:

- Control Condicional: Las instrucciones de bifurcación condicional realizan una prueba y una bifurcación solo si la prueba es VERDADERA.
- Control Incondicional: Las instrucciones de bifurcación incondicionales, llamadas saltos, siempre bifurcan.

2.1- Estructuras de Control - Condicional

Todos los procesadores incluyen operaciones condicionales que permitan ejecutar una serie de instrucciones u otras en función de un resultado lógico.

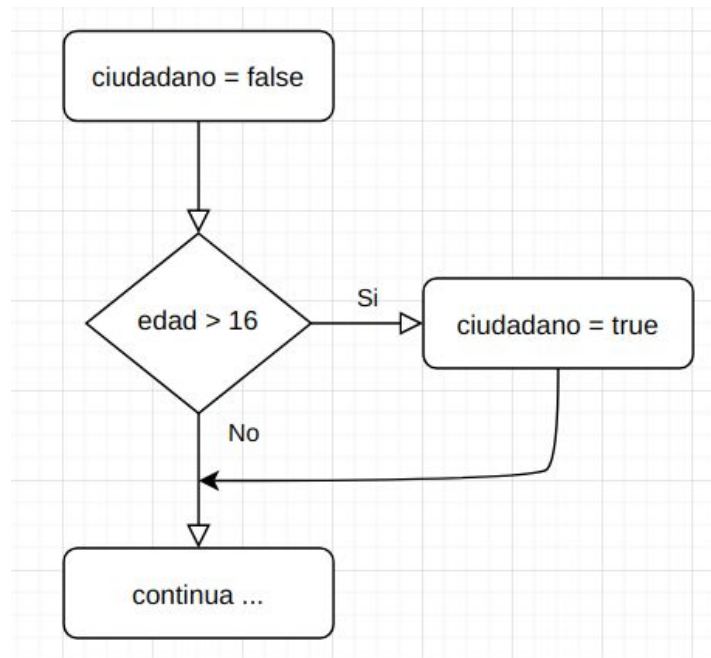
- Esta toma de decisiones se representa en los lenguajes de programación mediante sentencias: if, if/else, switch/case.
- En otros casos la toma de decisiones permite repetir código mediante sentencias: while, for.

2.1.1 Ejemplo decisión de un IF

Dado el siguiente código en lenguaje de alto nivel, codifique en assembler.

```
ciudadano = false;  
If (edad > 16) then  
    ciudadano = true;  
end if;
```

```
addi t0, zero, 16  
add t1, zero, zero  
blt t1, t0, fin      # note qué la condicion es opuesta  
addi t1, zero, 1  
fin: ...
```



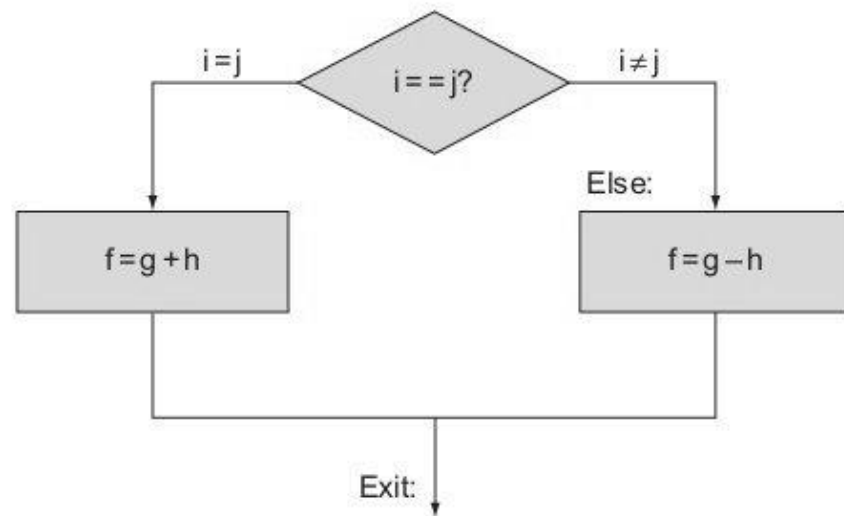
2.1.2 Ejemplo decisión de un IF/ELSE

Dado el siguiente código en lenguaje de alto nivel, codifique en assembler.

```
if (i==j) then
    f = g+h;
else
    f = g-h;
end if;
```

Suponga que f, g, h, i, j están cargados en los registros: x19, x20, x21, x22, x23.

```
bne x22,x23,else
add x19,x20,x21
beq x0,x0,fin
else: sub x19,x20,x21
fin:  ...
```



2.1.3 Ejemplo decisión de un SWITCH/CASE

```
switch (button) {  
    case 1: amt = 20; break;  
    case 2: amt = 50; break;  
    default: amt = 0;  
}
```

```
# s0 = button, s1 = amt  
case1:  
    addi t0, zero, 1    # t0 = 1  
    bne s0, t0, case2   # saltar si (button != 1)  
    addi s1, zero, 20   # if =, amt = 20  
    j done              # salir del caso  
case2:  
    addi t0, zero, 2    # t0 = 2  
    bne s0, t0, default # saltar si (button != 2)  
    addi s1, zero, 50   # if =, amt = 50  
    j done  
default:  
    add s1, zero, zero # amt=0  
done:
```

2.1.4 Ejemplo repetición de un WHILE

```
// determinar la potencia de x tal que  $2^x$   
= 128  
int pow = 1;  
int x = 0;  
while (pow != 128) {  
    pow = pow * 2;  
    x = x + 1;  
}
```

```
# s0 = pow, s1 = x  
addi s0, zero, 1    # pow = 1  
add s1, zero, zero  # x = 0  
addi t0, zero, 128  # t0 = 128  
while: beq s0, t0, done # pow = 128?  
    slli s0, s0, 1    # pow = pow * 2  
    addi s1, s1, 1    # x = x + 1  
    j while          # repetición  
done:
```

2.1.5 Ejemplo repetición de un DO/WHILE

```
// determinar la potencia de x tal que  $2^x$   
= 128  
int pow = 1;  
int x = 0;  
do {  
    pow = pow * 2;  
    x = x + 1;  
} while (pow != 128);
```

```
# s0 = pow, s1 = x  
    addi s0, zero, 1      # pow = 1  
    add s1, zero, zero    # x = 0  
    addi t0, zero, 128    # t0 = 128  
while: slli s0, s0, 1     # pow = pow * 2  
    addi s1, s1, 1        # x = x + 1  
    bne s0, t0, while     # pow = 128?  
done:
```

2.1.6 Ejemplo repetición de un FOR

```
// sumar numeros del 0 al 9
int sum = 0;
int i;
for (i = 0; i < 10; i = i + 1) {
    sum = sum + i;
}
```

```
# s0 = i, s1 = sum
    addi s1, zero, 0    # sum = 0
    addi s0, zero, 0    # i = 0
    addi t0, zero, 10   # t0 = 10
for: bge s0, t0, done   # i >= 10?
    add s1, s1, s0      # sum=sum + i
    addi s0, s0, 1      # i = i + 1
    j for               # repeticion
done:
```

2.2 Estructuras de Control - Incondicional

Un programa puede saltar, es decir, bifurcarse incondicionalmente, usando una de 2 instrucciones o 3 pseudoinstrucciones:

- Saltar y enlazar (*jal*), 20
- Saltar y enlazar al registro (*jalr*), 12
- Saltar (*j*) pseudo jal,
- Saltar registro (*jr*) pseudo jalr
- Retorno (*ret*) pseudo jalr.

2.2.1 Ejemplo salto incondicional

Para salir del case usamos break.

```
switch (button) {  
    case 1: amt = 20; break;  
    case 2: amt = 50; break;  
    default: amt = 0;  
}
```

*En asm usamos **j** etiqueta.*

Tambien lo vimos en While y For

s0 = button, s1 = amt

case1:

```
    addi t0, zero, 1    # t0 = 1  
    bne s0, t0, case2   # saltar si (button != 1)  
    addi s1, zero, 20   # if =, amt = 20  
    j done             # salir del caso
```

case2:

```
    addi t0, zero, 2    # t0 = 2  
    bne s0, t0, default # saltar si (button != 2)  
    addi s1, zero, 50   # if =, amt = 50  
    j done
```

default:

```
    add s1, zero, zero # amt=0
```

done:

Preguntas ?